

Ops 1: Manual Minecraft Server Runbook

Name: Dean Leon | Student ID: 934513949

1. Infrastructure Overview

Manual deployment of a persistent PaperMC Minecraft server on AWS Academy.

Hardware Specifications

- **Instance Type:** `t3.small` (2 GiB RAM).
- **AMI:** Ubuntu Server 24.04 LTS.
- **Storage:** 12 GiB EBS gp3.

2. System Preparation & User Setup

Create a dedicated system user to isolate the application process for security

```
sudo useradd -r -m -d /opt/minecraft -s /bin/bash minecraft
```

Update system repositories and install the required Java 21 runtime

```
sudo apt update  
sudo apt install openjdk-21-jre-headless -y
```

3. Minecraft Server Installation

Create the application directory and assign ownership to the service user

```
sudo mkdir -p /opt/minecraft/server  
sudo chown -R minecraft:minecraft /opt/minecraft
```

Switch to the service user and navigate to the server folder

```
sudo su - minecraft  
cd ~/server
```

Download the PaperMC server software

```
curl -o paper.jar https://api.papermc.io/v2/projects/paper/versions/1.21.1/builds/130/downloads/paper-1.21.1-130.jar
```

Run the server once to generate configuration files and then accept the EULA

```
java -Xms1G -Xmx1G -jar paper.jar --nogui  
vim eula.txt
```

4. Configuration & RCON (Extra Credit)

Modify server settings for identity (MOTD) and remote administration (RCON)

```
vim /opt/minecraft/server/server.properties
```

Install build tools and compile the mcrcon utility from source

```
sudo apt install git build-essential -y  
git clone https://github.com/Tiiffi/mcrcon.git  
cd mcrcon  
make  
sudo make install
```

5. Service Management (Systemd)

Create the background service configuration file

```
sudo vim /etc/systemd/system/minecraft.service
```

Reload the systemd daemon, enable the service for boot, and start it

```
sudo systemctl daemon-reload  
sudo systemctl enable minecraft.service  
sudo systemctl start minecraft.service
```

6. Operational Procedures

Check if the server is active and running

```
sudo systemctl status minecraft.service
```

Monitor the live server logs for errors or player connections

```
sudo journalctl -u minecraft.service -f
```

Execute an administrative command while the server is running in the background

```
mcrcon -p admin "op [MINECRAFT_USERNAME]"
```

Security Note: RCON (25575) is restricted to `localhost`. SSH is required for administration.

7. Cost-Control Strategy

Instance Size Choice & Rationale:

A `t3.small` instance was selected for this deployment. While AWS Academy provides smaller instances, PaperMC usually requires a larger amount of ram, like 2GB of RAM to run Java 21 comfortably. Choosing a `t3.small` prevents OOM errors and kernel panics.

Stop/Shutdown Schedule:

To preserve lab credits, the instance follows a strict on demand schedule. The server is manually started at the beginning of a development or testing session and is shut down via the AWS Console immediately upon completion of the task. The server is never left running overnight or during periods of inactivity.

Cost Guardrails:

To prevent accidental credit exhaustion, I utilize **lab session discipline**. This involves setting a timer for each work session and verifying the instance state in the AWS Console before closing the browser. Additionally, the AWS Academy environment's built-in session timer acts as a hard guardrail to automatically terminate/stop resources if a session is accidentally left open.

Document Attribution:

I used LLM assistance to format this document and to ensure that all CLI commands are correct.

<https://github.com/CS-312-001-S2026/minecraft-ops-GenericProgram>